



Iterator

Juan Esteban García Botero-Jeronimo García Velásquez | Ingeniería de sistemas

Propósito

- Permite recorrer los elementos de una colección sin exponer cómo está construida por dentro (lista, pila, árbol, etc.)
- Idea clave: separar qué se recorre de cómo se recorre

Problema

- Una colección es un contenedor de objetos, y puede estar basada en listas, pilas, árboles o grafos
- Toda colección debe ofrecer una forma de acceder a sus elementos
- Recorrer una lista es fácil, pero un árbol admite varios recorridos (en profundidad, en anchura, acceso aleatorio)
- Meter todos esos algoritmos en la colección desvía su responsabilidad principal: almacenar datos de forma eficiente
- El cliente termina acoplado a cada tipo de colección concreta

La solución

- Extraer el comportamiento de recorrido y ponerlo en un objeto aparte: el iterador
- El iterador guarda el estado del recorrido (posición actual, elementos restantes)
- Varios iteradores pueden recorrer la misma colección a la vez sin interferir
- Todos los iteradores implementan la misma interfaz, así que el cliente no depende de la colección ni del algoritmo
- Para un recorrido nuevo, se crea una nueva clase iteradora sin tocar la colección ni el cliente

Analogía del mundo real

Visitar Roma: tres formas de recorrer las atracciones

- Ir improvisando con direcciones en la cabeza
- Usar una app de guía en el smartphone
- Contratar a un guía local

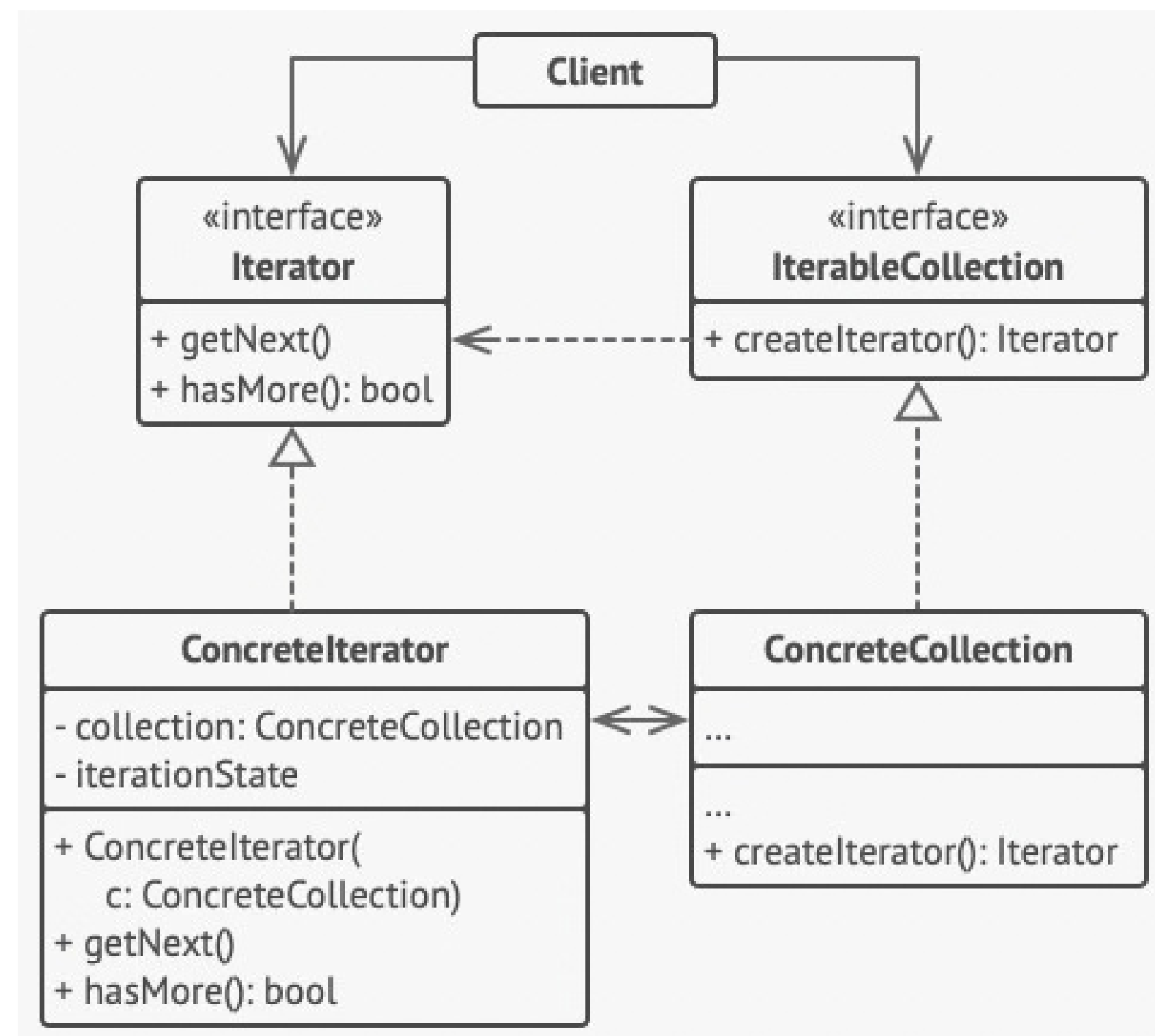
Las tres actúan como iteradores sobre la misma "colección" de lugares



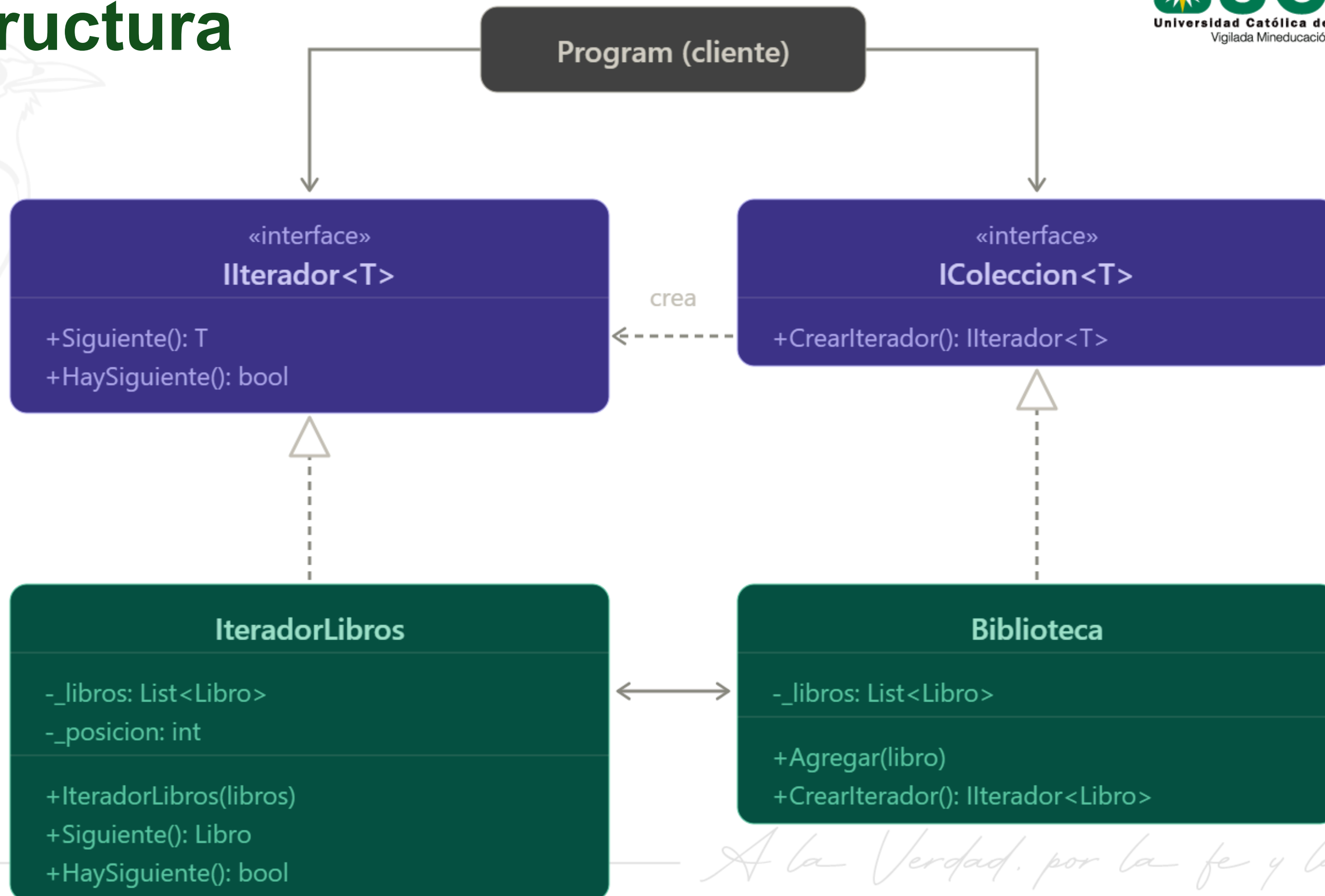
A la Verdad, por la fe y la ciencia

Estructura

1. Iterator (interfaz): declara las operaciones de recorrido (siguiente, posición actual, reiniciar)
2. Iteradores concretos: implementan algoritmos específicos y llevan su propio progreso
3. Colección (interfaz): declara el método que entrega iteradores compatibles
4. Colecciones concretas: devuelven una instancia nueva de su iterador cada vez que se les pide
5. Cliente: trabaja solo con las interfaces, sin acoplarse a clases concretas



Estructura



A la Verdad, por la fe y la ciencia

¿Cuándo usarlo?

- Cuando la colección tiene una estructura interna compleja que quieres ocultar al cliente, por comodidad o por seguridad
- Para reducir código de recorrido duplicado en la aplicación
- Cuando el código debe funcionar con distintas estructuras de datos, o con tipos que no se conocen de antemano

Cómo implementarlo (5 pasos)

1. Declarar la interfaz iteradora (mínimo: obtener el siguiente elemento)
2. Declarar la interfaz de colección con un método que devuelva iteradores
3. Implementar los iteradores concretos, cada uno ligado a una sola instancia de colección
4. Implementar la interfaz de colección; la colección se pasa a sí misma al constructor del iterador
5. Reemplazar en el cliente todo el código de recorrido por el uso de iteradores

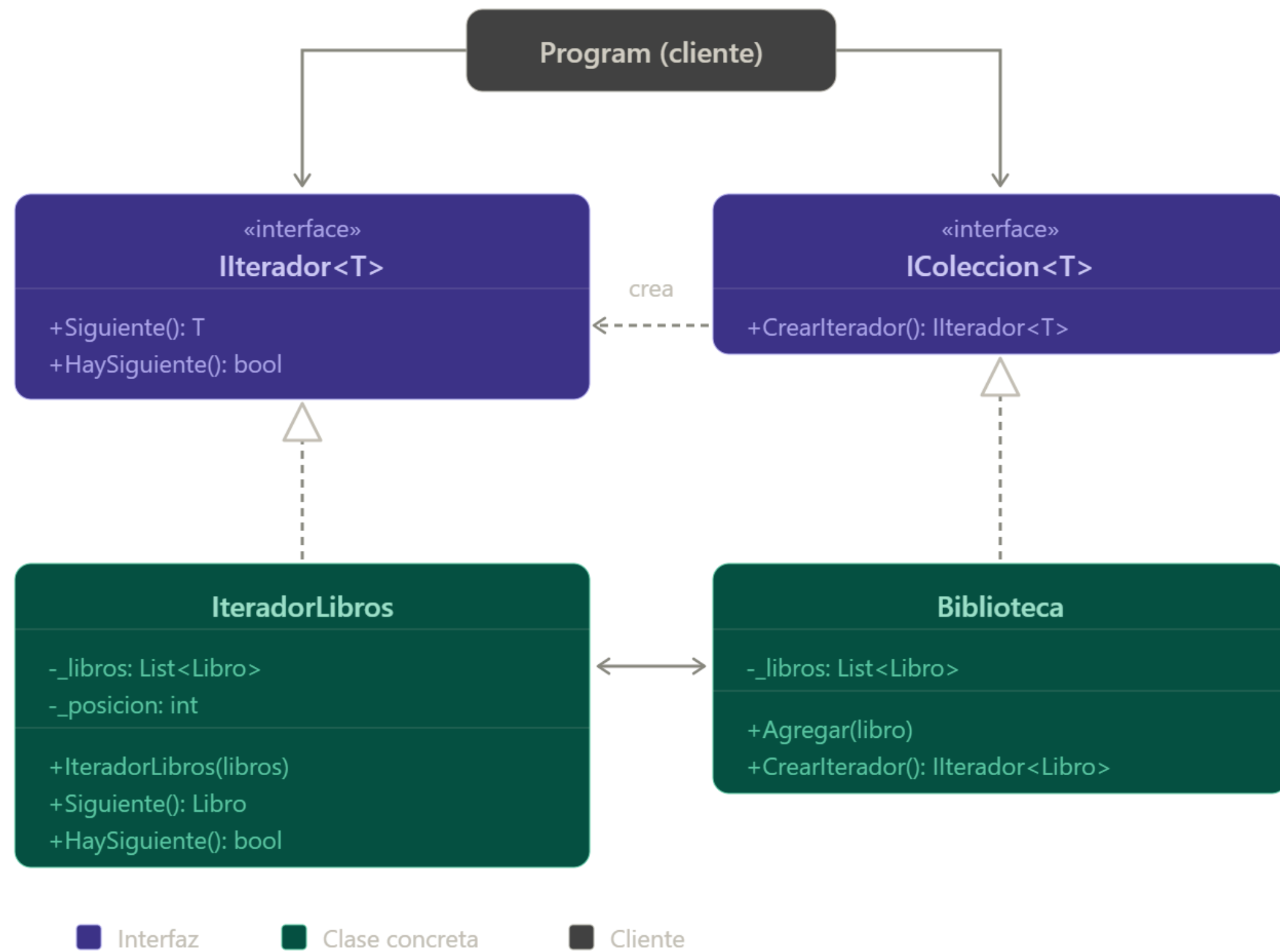
Pros y contras

Ventajas:

- Principio de responsabilidad única: el recorrido sale de la colección y del cliente
- Principio abierto/cerrado: se añaden colecciones e iteradores sin romper código existente
- Recorridos en paralelo, porque cada iterador tiene su propio estado
- Se puede pausar una iteración y retomarla después

Desventajas:

- Puede ser excesivo si solo trabajas con colecciones simples
- Puede ser menos eficiente que recorrer directamente algunas colecciones especializadas



Referencias

- Refactoring.Guru. (s. f.). Iterator. <https://refactoring.guru/es/design-patterns/iterator>
- Composite: <https://refactoring.guru/es/design-patterns/composite>
- Factory Method: <https://refactoring.guru/es/design-patterns/factory-method>
- Memento: <https://refactoring.guru/es/design-patterns/memento>
- Visitor: <https://refactoring.guru/es/design-patterns/visitor>



¡Gracias!